Edge AI→ Advanced Embedded Systems

Tony Genev

My First Machine Learning Challenge Report

Overview

In this project, I explored my first hands-on experience with machine learning using Python, implementing a k-Nearest Neighbors (k-NN) classifier on three datasets: Iris, Wine, and Computer Hardware.
The goal was to understand the full workflow of supervised learning, from data preparation to model evaluation and predictions.

Key Steps in the Project

1. **Data Preparation and Cleaning**
   I loaded the datasets, inspected their structure, and cleaned the data to ensure it was ready for analysis.

```python
# two example datasets
df_iris = pd.read_csv("http://archive.ics.uci.edu/ml/machine-learning-databases/iris/iris.data")
df_wine = pd.read_csv("http://archive.ics.uci.edu/ml/machine-learning-databases/wine/wine.data")

# the Computer Hardware dataset will be left as an exercise
df_machine = pd.read_csv("https://archive.ics.uci.edu/ml/machine-learning-databases/cpu-performance/machine.data", header=None)
```
✓ 1.2s

Iris

```python
df_iris['Species-id'] = df_iris['Species'].map({'Iris-setosa': 0, 'Iris-versicolor': 1, 'Iris-virginica': 2})
df_iris.head(5)
```
[6] ✓ 0.0s

	Sepal Length	Sepal Width	Petal Length	Petal Width	Species	Species-id
0	4.9	3.0	1.4	0.2	Iris-setosa	0
1	4.7	3.2	1.3	0.2	Iris-setosa	0
2	4.6	3.1	1.5	0.2	Iris-setosa	0
3	5.0	3.6	1.4	0.2	Iris-setosa	0
4	5.4	3.9	1.7	0.4	Iris-setosa	0

Figure 1: Data Preparation and Cleaning Steps

2. **Data Exploration**

Visualizing the data helped me understand relationships between features and the target variable. Scatter plots and box plots revealed patterns and outliers.

```python
# visualization of possible class/target column
machine_features = ['MYCT', 'MMIN', 'MMAX', 'CACH']

df_machine.boxplot(column=machine_features,by='PRP_Class',figsize=(15,5),layout=(1,4)
);
```

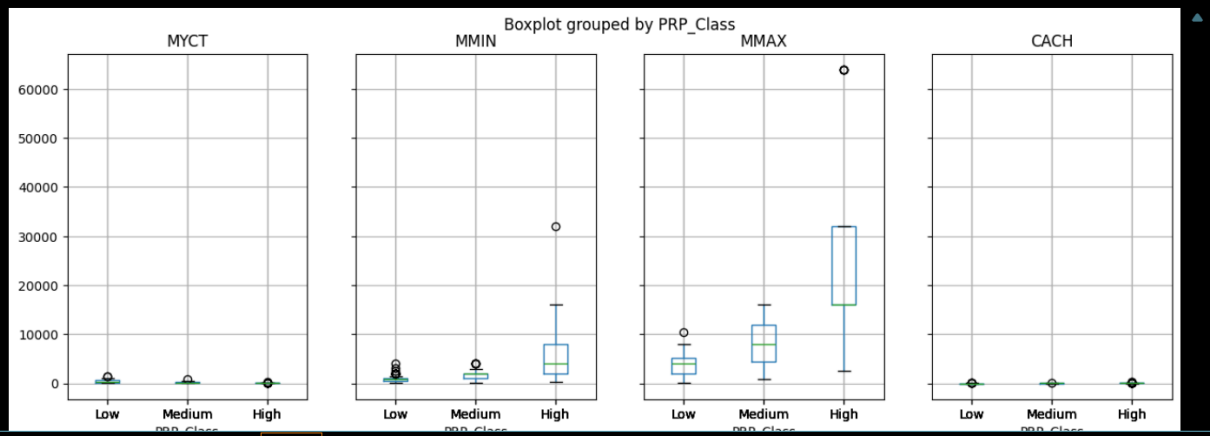


Figure 2: Data Exploration through Visualization

3. **Feature Selection and Scaling**

I selected the most relevant features and scaled them to improve the k-NN model's performance.

```python
X_machine = scaler_machine.transform(X_machine)
print('The length of X_machine: {}'.format(len(X_machine)))

# Scatter plot
vendor_to_num = {vendor: i for i, vendor in enumerate(y_machine.unique())}
y_machine_num = y_machine.map(vendor_to_num)
plt.scatter(X_machine[:,0], X_machine[:,1], edgecolors='k', c=y_machine_num);
```

```
classification possibilities: ['adviser' 'amdahl' 'apollo' 'basf' 'bti' 'burroughs' 'c.r.d' 'cdc'
 'cambex' 'dec' 'dg' 'formation' 'four-phase' 'gould' 'hp' 'harris'
 'honeywell' 'ibm' 'ipl' 'magnuson' 'microdata' 'nas' 'ncr' 'nixdorf'
 'perkin-elmer' 'prime' 'siemens' 'sperry' 'sratus' 'wang']
The length of X_machine: 209
```

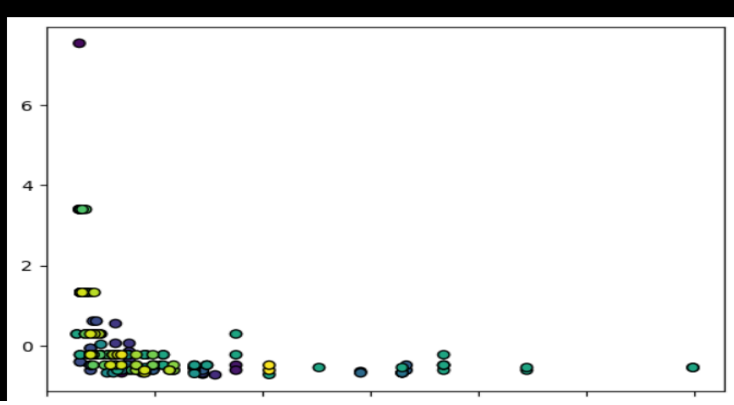


Figure 3: Feature Selection and Scaling

4. **Model Training**

The k-NN model was trained using the training dataset. I experimented with different values of k to see their effect on accuracy.

```
# For example, let's consider two new flowers,
# one with Petal length = -0.9, Petal Width = -1  (scaled values)
# and with Petal length = 1, Petal Width = 1.3  (scaled values)

some_flowers_X = [[-0.9,-1], [1,1.3]]

# The model predicts..
some_flowers_y = clf_iris.predict(some_flowers_X)
print("The predicted species are\n \
    flower 1: Species_id = {0}\n \
    flower 2: Species_id = {1}\n".format(some_flowers_y[0],some_flowers_y[1]))
```

```
The predicted species are
    flower 1: Species_id = 0
    flower 2: Species_id = 2
```

Figure 4: Training the k-NN Model

5. **Model Evaluation**

I evaluated the model's performance on the test dataset using accuracy scores and examined how well it generalized to unseen data.

```
#Evaluate performance (with test set)
print('Number of test points: ',X_test_machine.size)
y_pred_machine = clf_machine.predict(X_test_machine)
metrics.accuracy_score(y_test_machine, y_pred_machine)
```

```
Number of test points:  212

0.1509433962264151
```

Figure 5: Model Evaluation Results

6. **Making Predictions**
   Finally, I applied the trained model to new data points to see its predictions in action.

```
#Apply different models to some new machines
some_machines_X = [[0.90, 0.2, -0.4, -0.16], [7.2, -1.0, -0.99, -0.6]]
some_machines_y = clf_machine.predict(some_machines_X)
print("The predicted machine vendors are\n \
    machine 1: Vendor Name = {0}\n \
    machine 2: Vendor Name = {1}\n".format(some_machines_y[0],some_machines_y[1]))
```

```
The predicted machine vendors are
    machine 1: Vendor Name = wang
    machine 2: Vendor Name = ibm
```

Figure 6: Predictions Made by the Model

Key Learnings

- Understanding k-NN - Learned how majority voting among neighbors determines predictions.

  In Canvas more information is provided about kNN. Read through it and write down how you think the learning works.

  kNN algorithm or k-closest Nearest Neighbors algorithm is used in classification to predict a target whether should go to class "A" or class "B". The algorithm uses k (the number of all neighbors that should be taken to account) and then the distances from the neighbors to the target. Moreover kNN is like a voting system, where the majority is what decides on what should the target be. So if the majority of the neighbours are apples and not melons, than the target should be an apple(k is almost like a range of the votes). Also the k is good to be an odd number so that we don't end up with a tie, and its good to be a high number but not too high, as it can become to generic.

- **Data Handling** - Practiced cleaning, transforming, and manipulating datasets with Pandas.

- **Visualization** - Learned how plotting features and relationships helps in feature selection and model understanding.

- **Model Evaluation** - Learned to split data into training and test sets and interpret accuracy scores effectively. (80% training and 20% testing)

Challenges Faced

During this project, I faced several challenges that required careful attention. For data cleaning, the Computer Hardware dataset contained textual information like vendor names, which I converted into numerical values using mapping so the model could process them. For feature selection, I had to decide which features, such as CPU speed or memory size, were most relevant to include, based on understanding how they relate to the target variable. Finally, model evaluation was challenging because interpreting accuracy scores correctly to assess the model's performance required practice and careful analysis.

Conclusion

All in all, this project was a good introduction to machine learning, providing hands-on experience with the k-NN algorithm and the complete workflow: Preparation → Exploration → Training → Evaluation → Prediction. It helped me develop practical skills in data cleaning, feature selection, model training, and evaluation, while also improving my ability to interpret results and make informed decisions. Additionally, it prepared me for working with platforms like Edge Impulse, giving me a solid understanding of how AI models are trained, tested, and applied to real-world data. Overall, this experience strengthened both my technical abilities and my confidence in approaching more advanced machine learning projects in the future.

Part 2 Edge AI on Edge Impulse

Edge AI explanation

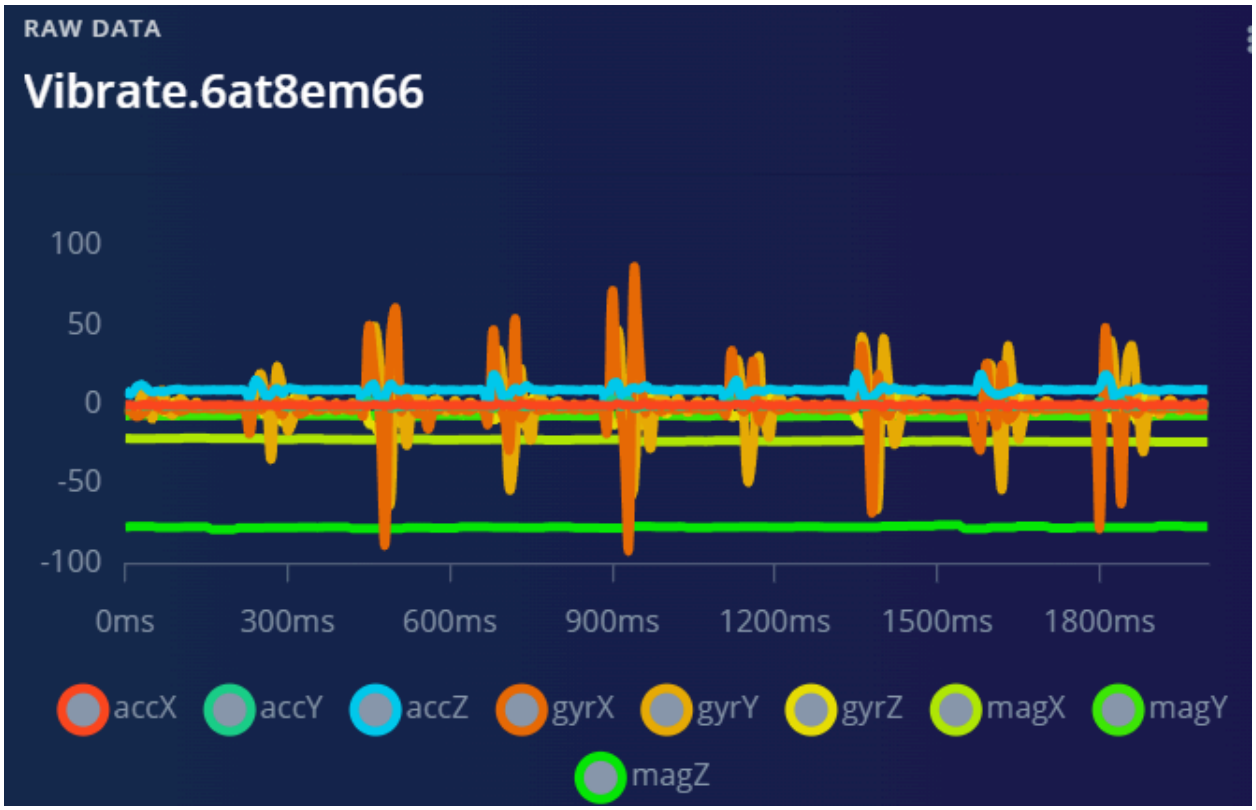
From what I researched and learned, edge ai is the process of using an ai algorithm inside a device (like a microcontroller or phone) instead of using the cloud, because this saves energy but also makes it more secure and faster. This of course is because we don't have to constantly communicate with the cloud for certain actions. However, that doesn't mean to train the model on the device, therefore we have to train it on the cloud and after that clustering it in the device which will indeed sacrifice some of the accuracy, but not significantly. Moreover, in order for the ai model to be ran on the device I need to make a set of data, clean it, split it in 80% training and 20% testing, and then train it and after that cluster it so that I can upload it on the device.

Edge Impulse

Edge Impulse is a well-known edge AI platform for collecting data, training models, and deploying them to your edge computing devices. It provides an end-to-end framework that easily plugs into your edge device. I just had to download the edge impulse CLI and then run the command "edge-impulse-daemon" and now I connected the arduino nano 33ble.

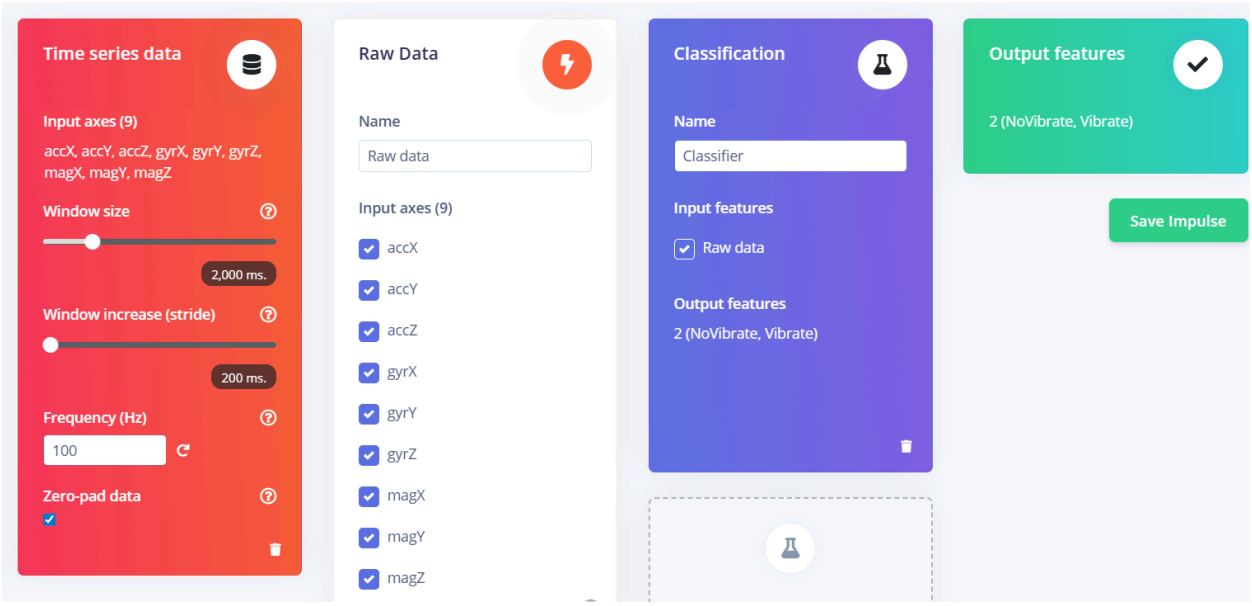
Data Collection

I started my dataset by making two classes (labels) → “Vibarte” and “NoVibrate”, and start with some “NoVibrate” samples for a few seconds where I don’t do anything to the environment of the arduino and then for “Vibrate” I just started banging on the table so that I make a vibrating environment for the arduino.



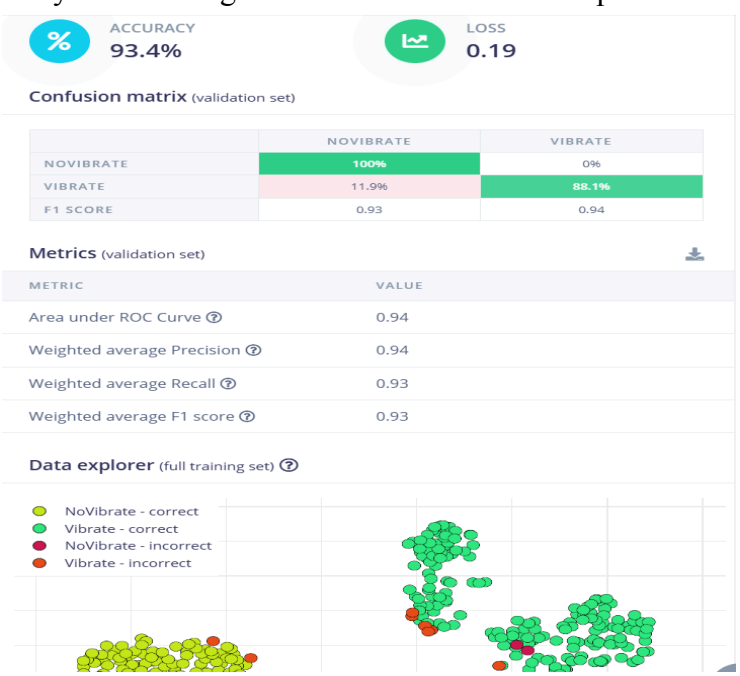
Impulse Creation

I first start with making the window size, where in my case is 2000 ms, but that doesn’t mean it will stop analyzing the data after those 2000 ms, but it will do it in segments. Then I have to decide on a processing block, where I have a few options and I chose to go for raw data for my first impulse where all the data on the arduino is taken into account from the gyroscope. After that I have to choose a learning block, and for now I choose using regular classification. In the end, I have two output features: “Vibrate” and “NoVibrate”. For my second impulse I experimented with Spectrogram, which I saw that is for audio, however I can choose to use one of the data in my case I used gyrX, however this is not that efficient as there can be more to it than just one set of variables. So in the end for Impulse 2 I got 100% accuracy, but this is not as reliable as the first, because of the use of one variable.



Training the Model

I have to train the model, and choose to train it on a GPU or CPU, where the GPU is faster, however the CPU is good enough for the small dataset that I have so I have to go with that. Also I can choose the training cycles in my case 30. After that I have to retrain the model, here everything is done for me by edge impulse, but the interesting part after is the fact that I can see how accurate my model is before putting it on the device itself, so before the clustering, in my case the accuracy was 93.4% and 0.19 loss. I can also see what is the success or accuracy of every class. This gives a clear overview of the performance before deployment.



Model Testing

Before the testing of the model, I have an option to live classify where I will generate new data and the model will determine the class of this data, if it goes wrong I can correct it of course, this live classification can then be added to the ever growing dataset that I have. Furthermore, I can test the model with the 20% data that I made specifically for testing, so now I can see the real model accuracy before deployment, of course this is the accuracy of the cloud model. The model takes the data and blindly decides on the classes, if there is a mistake in the end the model calculates that as the final accuracy percentage(in my case 81.82%).

SAMPLE NA...	EXPECTED OUT...	LENG...	ACCURACY	RESULT
Vibrate.6a...	Vibrate	5s	31%	8 NoVibrate, 5 Vibr...
Vibrate.6a...	Vibrate	5s	100%	16 Vibrate
NoVibrate....	Vibrate	2s	0%	1 NoVibrate
NoVibrate....	NoVibrate	2s	100%	1 NoVibrate
NoVibrate....	NoVibrate	2s	100%	1 NoVibrate
NoVibrate....	NoVibrate	2s	100%	1 NoVibrate
testing.6at...	Vibrate	10s	95%	39 Vibrate, 2 uncer...

```
Job started at 27 Nov 2025 13:16:20
INFO: Created TensorFlow Lite XNNPACK delegate for CP
Classifying data for Classifier OK

Generating model testing summary...

Job completed (success)
```

Results Model version

ACCURACY
81.82%

Metrics for Classifier

METRIC	VALUE
Area under ROC Curve	0.91
Weighted average Precision	0.97
Weighted average Recall	0.83
Weighted average F1 score	0.88

Deployment of the Model

In the end, I can build the model and optimize (cluster) it so that it fits on the Arduino Nano 33 BLE. Edge Impulse converts the neural network into C++ code and reduces its size while keeping almost the same performance. After this step, I can see the actual accuracy, flash usage, RAM usage, and latency of the model on the device itself. This shows how well it will run in real time on the microcontroller and confirms that the optimized model still performs reliably.

Quantized (int8)

Selected ✔

	RAW DATA	CLASSIFIER	TOTAL
LATENCY	-	2 ms.	**2 ms.**
RAM	7.0K	3.1K	**7.0K**
FLASH	-	49.6K	**-**
ACCURACY			**80.52%**

Unoptimized (float32)

Select

	RAW DATA	CLASSIFIER	TOTAL
LATENCY	-	6 ms.	**6 ms.**
RAM	7.0K	8.4K	**8.4K**
FLASH	-	152.4K	**-**
ACCURACY			**81.82%**

Conclusion

All in all, I learned how to collect data, extract useful features, train a neural network, and deploy it onto the Arduino Nano 33 BLE using Edge Impulse. Before deployment, I could clearly see the model's accuracy and loss, and after optimizing it for the device, I could check how it performs in terms of memory, speed, and accuracy on the actual hardware.
I also understood why quantization is useful. Quantization converts the model from float32 values to smaller 8-bit integers. This makes the model much smaller, faster, and more energy-efficient, which is perfect for microcontrollers like the Nano 33 BLE that have limited memory. The important part is that quantization keeps the accuracy almost the same while allowing the model to fit on the device.